

Exacube Assignment

Submission: within 2–3 days of receiving this brief. **Dataset:**

<https://www.kaggle.com/datasets/samyakrajbayar/synthetic-discord-analytics-dataset>

Overview

Build a full-stack analytics application over a synthetic Discord activity dataset: a Postgres-backed FastAPI service, a frontend, and — the heart of this — a conversational agent that answers questions about the data by writing SQL, and can also **produce artifacts**: charts the user pins to a dashboard, Excel workbooks, PowerPoint decks.

Those artifact-producing capabilities must be built as **plugins**. We should be able to drop a new plugin into the codebase and have the agent pick it up and use it, without touching the agent's core, editing a router, or hand-editing a prompt.

The plugin architecture is what we are actually assessing. The dataset is a means to an end. A submission with a beautiful schema and a hardcoded `if tool == "chart"` chain will score badly; a submission with an adequate schema and a genuinely extensible tool layer will score well.

Before you start: read the whole brief, then reply with (a) your estimate of the hours you need, (b) the day you intend to submit, and (c) anything you plan to deliberately cut and why.

This is more work than fits comfortably into 2–3 days, and that is intentional. **Scoping is part of the assessment.** We would far rather see a coherent subset built properly, with the cuts named and defended in your README, than everything half-finished. Choose deliberately, tell us what you chose, and hold to it. If the plan changes along the way, say so — a revised estimate with a reason is a good signal; silence is not.

Part 1 — Data foundation (keep this lean)

Enough to support the rest. Don't gold-plate it.

- Load the dataset into Postgres via a **reproducible, idempotent script or migration** — not a manual import. Running it twice must not duplicate rows.
- Design a sensible relational schema over whatever entities the data gives you — servers, channels, members, messages, events, and so on. Real types, foreign keys, timestamps stored as timestamps. We'll ask you to defend it briefly; we won't spend the call there.
- This is **event-log-shaped data**, and that has consequences: time-bucketed queries will dominate, so index for them. Be explicit about how you handle time zones, and about the grain of each table.
- The raw data will have inconsistencies. Handle them and say what you did — a few lines in the README is fine.

Budget roughly 15% of your time here.

Part 2 — API

FastAPI, serving the data and hosting the agent.

- Clean separation of routing, logic, and data access — we should be able to test the logic without HTTP.
 - Async, and honestly so: an async driver behind `async def`, not a sync driver blocking the event loop.
 - A handful of endpoints to power the frontend, including at least one **time-series aggregate** (activity per channel per day, member growth over time, hourly message distribution — your choice) computed **in the database**, not in Python.
 - Pydantic in and out. A consistent error envelope. Bad input yields a useful 4xx, never a 500 or a stack trace on the wire.
 - Tests in pytest against a **real Postgres** (testcontainers or a compose service — not SQLite, not mocks). Test the things most likely to break, including the unhappy paths. Coverage percentage is not the metric.
-

Part 3 — The agent and its plugin system

This is the assessment. Everything else is scaffolding for it.

The core loop

A chat interface where the user asks questions in natural language — “*which channels died after March?*”, “*who are the top 10 posters in the gaming server?*”, “*show me weekday vs weekend activity*”. The agent decides what to do, calls tools, and responds. Minimum behaviour:

- **Query the data.** Generate SQL, execute it, explain the result.
- **Recover.** If the SQL errors or a tool fails, the agent sees the failure and retries — bounded attempts, graceful surrender.
- **Decline.** “I can’t answer that from this dataset” is a correct answer, and we will test for it.
- **Chain.** “*Chart message volume per channel for the last quarter, then put it in a deck*” should work as one turn. Tools compose; the output of one is available to the next.

The plugin contract — the core deliverable

Every capability beyond raw SQL is a plugin. **Design a plugin interface** and implement the four below against it.

A plugin declares its own name, description, and input schema; the agent’s tool list and prompt are **derived from the registry at runtime**. Discovery should be automatic (decorator registration, entry points, directory scan — your choice, justify it). The test we will actually run:

We will write a fifth plugin ourselves, following only your README, and drop it in. If it requires editing any file in your agent core, the design has failed. Time us — the closer to fifteen minutes, the better you’ve done.

Think about what the interface owes the system beyond `execute()` : validation of the LLM’s arguments, structured errors the agent can reason about and retry against, whether a plugin can stream progress, whether it can consume another plugin’s output, and how it declares that. Some of these you may reasonably decide not to support — say so, and say why.

The four plugins:

1. **query** — natural language to SQL, executed, results returned as structured data other plugins can consume.
2. **chart** — takes a result set and a spec, produces a chart. The agent chooses the chart type, or the user asks for one. This data is heavily temporal, so time series is the common case — but a top-N bar chart and a distribution should work too. Whether you return a spec the frontend renders, or a rendered image, is your call — argue for it.
3. **excel** — an `.xlsx` from one or more result sets. Real formatting, real headers, multiple sheets when the data calls for it. A dumped CSV with an `xlsx` extension is not this.
4. **powerpoint** — a `.pptx`. “*Give me a five-slide engagement summary for the top three servers*” should yield a deck with titles, a chart or two, and prose the agent wrote.

Downloadable artifacts get served properly — a real endpoint, sane content types, and some story about lifecycle and cleanup.

Pinning

Charts produced in chat can be **pinned to a dashboard** and persist across reloads. Pinned charts hold their underlying query, so a pinned chart is re-runnable, not a dead PNG. Users can unpin and reorder. This is a small feature that quietly forces you to decide what a chart *is* in your system — take it seriously.

Streaming

Real incremental streaming over SSE or WebSocket — not a blocking call behind a spinner, not a typewriter animation on a finished payload.

Stream the **stages**, not just tokens: the user should see the agent reasoning, then which tool it picked and with what arguments, then that tool’s progress, then results, then prose. Someone watching a deck get built should see it happening. Handle the client disconnecting mid-stream without leaking a connection, a query, or a half-written file.

Safety — non-negotiable

The agent generates SQL against your database and writes files to your disk. Assume the person in the chat box is hostile.

- The agent connects as a **read-only Postgres role**, grants scoped to exactly what it needs. Enforced at the database, not in the prompt.
- **Parse the generated SQL before executing it.** Single read statement, or reject. String-matching for `DROP` is not

validation and we will get past it.

- Statement timeout. Row cap.
- Artifact generation is a **resource-exhaustion surface**. This dataset has a lot of rows: “*export every message to Excel*” is one prompt away. A 400,000-row workbook, a 900-slide deck, fifty concurrent renders. Bound it.
- Assume **prompt injection, including via the data itself** — this is a dataset of *user-authored chat messages*, and the agent will be reading them back to itself. Someone’s username or message content is a plausible injection vector by construction. We consider this the most interesting attack surface in the brief. Tell us what you defended against and what you knowingly left open.

We will spend part of the follow-up call trying to break this.

Provider abstraction

The LLM sits behind an interface; any provider is fine. If you don’t want to spend your own credits, make it configurable and document exactly how we plug our key in — we will do that, and it must work first try.

Evaluation — we want a number

Build an eval harness. **At least 15 questions**, spanning: simple lookups, time-series aggregates, ambiguous phrasing (“*which channels are dying?*” — the agent has to pick a definition and state it), questions requiring a *chart*, questions requiring a *file*, at least one multi-tool chain, and at least three that are **not answerable** from the data.

Score two things separately: - **Routing** — did the agent pick the right tool(s)? - **Correctness** — was the answer right? (Result-set equivalence against a hand-written reference query beats string matching; if you disagree, argue it.)

Report the score in the README, with which categories fail and your honest read on why. A low honest number with a clear diagnosis beats a suspiciously high one with no methodology. We will re-run your harness. Log per-turn latency and token cost, and summarise.

Part 4 — Frontend

Consumes your API. Needs: a data table, a chart, the chat interface, and the pinned dashboard. **Visual design is not assessed.**

Behaviour is. The chat should render tool calls and their progress as they stream in. Loading, error, and empty states. A stream that dies halfway. A 500 from the API. A file that takes twenty seconds to generate.

Part 5 — Ship it

One command, clean checkout, works. `docker compose up` (or a documented `make up`) brings up database, backend, frontend, migrations, and the data load. A second documented command for the load is acceptable.

Also: - Multi-stage builds — not a 2GB image with the toolchain baked in. - Non-root containers. - Healthchecks, and dependency ordering that waits for *ready*, not *started*. - `.env.example` with every variable; startup fails loudly on missing config rather than 500ing at request time. - Structured JSON logs with a trace ID that follows a request through the API, into the agent, through each tool call, down to the SQL. - A `/health` endpoint that means something. - CI running lint, type check, and tests. Green.

Part 6 — Prove it holds up

Load-test the chat endpoint and the artifact-generation path (k6, Locust, whatever). **Report real numbers** — throughput, p50/p95/p99, error rate, where it fell over. Concurrent streaming connections and concurrent file renders are the interesting cases.

Then a **failure modes** section: the LLM provider times out or rate-limits you; a tool throws; Postgres disappears mid-turn; a user opens fifty streams; the model writes a cartesian join across the message table; a deck render blows out memory. For each — what happens today, and what you’d build for production.

Deliverables

1. **Git repository** (or zip) with all source.
2. **README.md** covering:

- How to run it. Assume Docker and nothing else.
 - Every environment variable and key we must supply.
 - **How to write a new plugin.** Treat this as documentation you'd ship. We will follow it literally.
 - **Design decisions and their tradeoffs.** Read closely. "I chose X" is worth nothing; "I chose X over Y, accepting cost Z, because of constraint C" is the point.
 - What you defended the agent against, and what you left open.
 - **Eval results** — routing score, correctness score, failures, diagnosis.
 - **Load test results** — the numbers, the bottleneck.
 - Failure modes.
 - **Estimated time, actual time, and where the difference came from.** An honest overrun with a clear cause reads far better than a suspiciously exact match.
 - What you'd do differently or additionally with more time. Specifics — this is where we learn whether you know what you didn't build.
3. Short **ADRs** (a paragraph each) for your biggest three or four calls. The plugin interface is certainly one of them.
-

How we assess

Roughly, in order:

1. **Does it run?** One command, clean machine. If not, the rest is academic.
2. **The plugin architecture.** Can we add a fifth plugin from your README without touching your core? This is the single largest weight in the assessment.
3. **The agent** — routing quality, tool chaining, safety, streaming, and whether your eval is honest.
4. **Judgement.** What you built well and what you deliberately left rough. Three parts done properly and one honestly descoped in the README beats five parts half-finished with no mention of it.
5. **Rigour.** Tests on the things that break. Numbers instead of adjectives.
6. **Communication.** The README, the ADRs, the honesty of the time report.

Expect a follow-up call walking through the code: why you structured the plugin layer that way, how it behaves under load and bad input, what its failure modes are, what you'd fix first. **We will write a plugin against your interface, and we will try to break your agent live.** Knowing exactly where it's weak is a strong answer. Claiming it isn't is not.

AI assistance in writing the code is fine and expected. Being unable to explain a line of your own submission is not.